pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

NumPy, Part 1

WEEK 2, DAY 1, PART 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of this part you can

Calculate on a whole column

Cost, income, and current yield for every bond.

```
costs = faces * prices / 100
```

Summarize a portfolio

Totals, averages, and a face-weighted average.

```
np.average(coupons, weights=faces)
```

Compare decimals safely

Two results that differ only in the last digit.

```
np.isclose(yields_a, yields_b)
```

Measure spread and returns

Standard deviation, and simple and log returns.

```
np.log(today / yesterday)
```

Week 2: the same four bonds, a new tool

The desk's four bonds, unchanged from week 1

Ticker	Coupon	Price	Score	Face held
QVNE	5.25	98.50	9	1,500,000
DNMR	6.125	103.25	11	500,000
PLWK	4.75	95.00	7	750,000
HLVF	7.50	101.00	14	250,000

Where lists run out: face times price

```
face_list * price_list
TypeError: can't multiply sequence
by non-int of type 'list'
```

A list holds numbers. It does not do arithmetic with them.

Week 1: a list and a loop

Four lines to say face times price over 100.

→

Today: a NumPy array

One line for the whole column.

→

Tomorrow: pandas

Built on top of NumPy.

A list repeats. An array calculates.

A list: * means repeat. No error, nothing doubled.

```
print(price_list * 2)
[98.5, 103.25, 95.0, 101.0,
 98.5, 103.25, 95.0, 101.0]
```

An array: * means multiply, bond by bond.

```
faces = np.array(face_list)
prices = np.array(price_list)
costs = faces * prices / 100
print(costs)
[1477500.  516250.  712500.  252500.]
```

`import numpy as np`

Loads the library and gives it a short name. Everyone who uses NumPy calls it np.

In Excel

`=B2*C2/100` filled down four rows
`=B2:B5*C2:C5/100` typed once, spills down

One array, one type

An array holds one type only, stored in dtype

Array	dtype	What it holds
`coupons`	`float64`	Numbers with a decimal point
`faces`	`int64`	Whole numbers
`tickers`	`<U4`	Text of up to 4 characters

Mix the types, and everything becomes text

```
mixed = np.array(['QVNE', 5.25])
print(mixed)
['QVNE' '5.25']
```

Arrays built from a pattern, with no list

Function	What it builds
`np.arange(95, 106, 1)`	95 to 105. The stop is left out.
`np.linspace(0.5, 7, 14)`	14 even steps. Both ends included.
`np.zeros(4)`	Four zeros, to fill in later
`np.ones(4) * 100`	A constant column of 100

Know the step: arange. Know the count: linspace.
A step with a decimal point? Prefer np.linspace().

One line, every bond

	QVNE	DNMR	PLWK	HLVF
faces	1,500,000	500,000	750,000	250,000
* prices	98.5	103.25	95.0	101.0
/ 100 = costs	1,477,500	516,250	712,500	252,500

```
costs = faces * prices / 100
print('Total cost:', costs.sum())
Total cost: 2958750.0
```

Python	Excel
`faces * prices / 100`	`=B2*C2/100 filled down`
`(faces * prices / 100).sum()`	`=SUMPRODUCT(B2:B5, C2:C5)/100`

Two averages, two different numbers

Each box is one bond, as wide as its share of the 3,000,000 of face: ticker, coupon, face held



Simple average

5.90625

Every bond counts the same.

```
np.mean(coupons)
```

Face-weighted average

5.458

163,750 of coupon on 3,000,000 of face.

```
np.average(coupons, weights=faces)
```

	Simple	Face-weighted
In Excel	=AVERAGE(D2:D5)	=SUMPRODUCT(D2:D5, B2:B5)/SUM(B2:B5)

Compare decimals with np.isclose(), not ==

The same yields, with the two steps in the other order

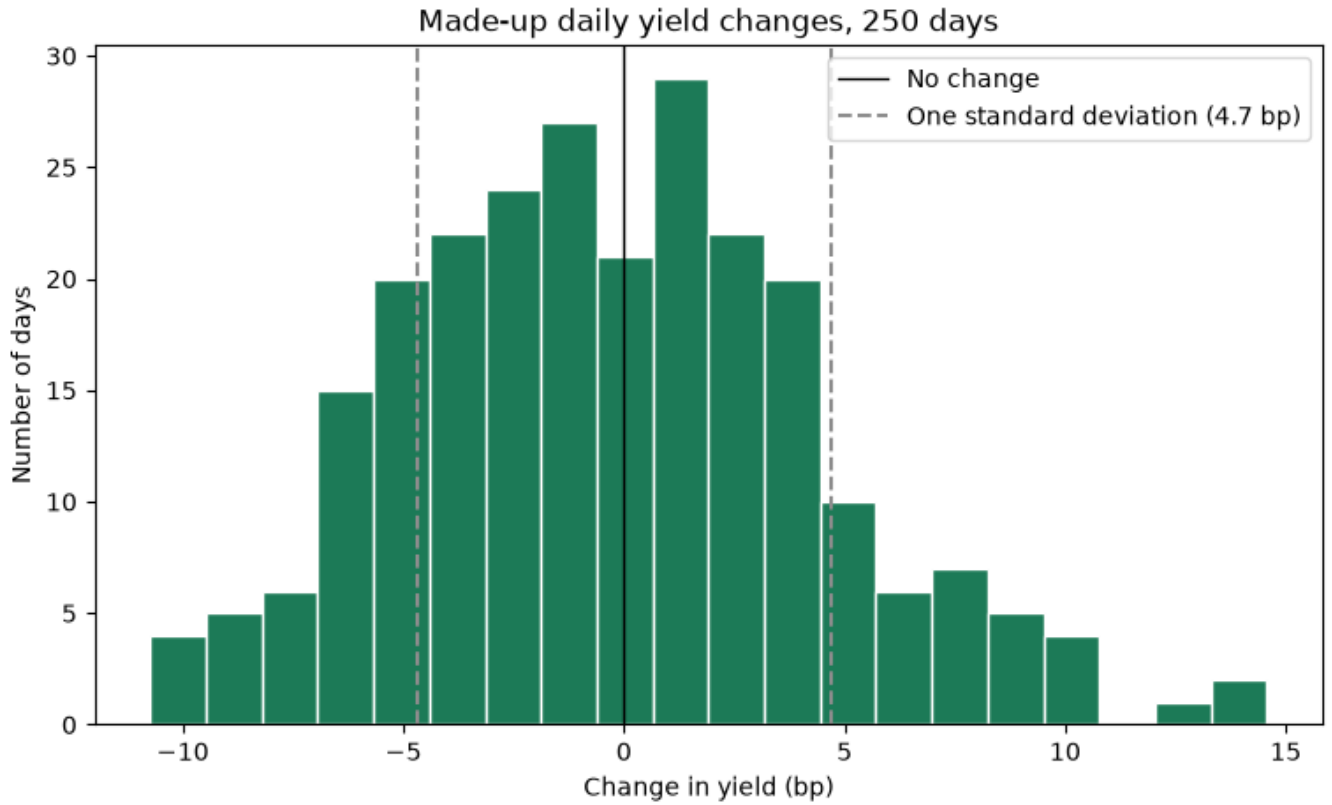
```
colleague_yields = coupons * 100 / prices
print(current_yields == colleague_yields)
[ True  True  True False]
print(np.isclose(current_yields,
                 colleague_yields))
[ True  True  True  True]
```

The two HLVF yields differ in the 15th decimal place. Both are 7.43 percent, and == still says False.

In Excel `=ABS(F2-G2)<0.000001` in place of `=F2=G2`

Numbers with a decimal point: np.isclose() or np.allclose(). Whole numbers and text: == is right.

Spread, random numbers, and returns



The seed

default_rng(42): the same numbers on every run

Two standard deviations

np.std(): the whole population. ddof=1: a sample, like STDEV.S

Log returns add up

np.log() over days sums to the whole move. Simple returns do not.

Read the picture and check it against the numbers in the notebook.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.

1

Read

Read the code before you run it.

2

Predict

Decide what it should produce.

3

Run

Run it and look at the output.

4

Check

Compare with a number you worked out yourself.

Code that runs is not the same as code that is right.

What you can do now

- Calculate cost, income, and current yield for every bond in one line
- Summarize a portfolio, down to a face-weighted average
- Compare decimal results with np.isclose()
- Measure spread, make random numbers with a seed, and work out returns

Next NumPy, Part 2: pick bonds out by a condition, and price a bond from its cash flows